Integration Plan: From a Pretrained LiDAR 3D Detector to a DSP-SLAM++-Style Multi-Class Object SLAM Pipeline

Planning document for `Code_tensor`

July 24, 2026

Abstract

This document plans the work required to grow the existing `lidar_detection` module — a trained PointPillars backbone with a CenterPoint detection head [11], using the `checkpoint_nuscenes_center.pth` weights — into a full object-aware SLAM system that reproduces the architecture of DSP-SLAM++ [6], itself an extension of DSP-SLAM [10]. It inventories what the current repository already provides, maps every remaining paper component to a concrete work package, sequences those work packages by dependency, and flags the architectural decisions and risks that should be resolved before implementation starts. No source code is modified by this document; it is a planning artifact only.

Contents

1 Scope and Starting Point **1**

2 Gap Analysis **2**

3 Target Architecture **3**

4 Work Packages **3**

5 Sequencing and Dependency Graph **7**

6 Architectural Decisions and Risks **7**

7 Proposed Repository Additions **8**

8 Summary **9**

1 Scope and Starting Point

The repository currently implements one block of the target system: the *3D detection candidate generator* ("Det3D" in Algorithm 1, line 2, of [6]). Concretely:

- `lidar_detection/models/pointpillars.py` — PointPillars [7] backbone; both an anchor head (`PointPillars`) and a CenterPoint heatmap head [11] (`CenterPointPillars`) are implemented. **This plan standardizes on the CenterPoint head**, per the project decision to use `checkpoint_nuscenes_center.pth`.

- `lidar_detection/detection/center_postprocess.py` — heatmap peak extraction (3×3 max-pool NMS, top-K per class) and box decoding for the CenterPoint head; this is the post-processing path relevant to this plan (the anchor-head path in `postprocess.py`/`anchor.py`/`nms.py` is not used going forward).
- `lidar_detection/pipeline.py` — `DetectionPipeline.run(points)` returns class-labeled 3D boxes `[x,y,z,l,w,h,yaw]` with scores, i.e. exactly the $d_{3D,i}$ tuples Algorithm 1 needs. **Note:** as currently written, `DetectionPipeline` hard-codes the anchor-head `PointPillars` model and the `postprocess` path; it does not yet know how to load `CenterPointPillars` or call `center_postprocess`. Wiring `DetectionPipeline` to the CenterPoint head/checkpoint is therefore the first concrete change needed and is captured as part of WP1 (Section 4).
- `lidar_detection/data/nuscenes_loader.py`, `train_nuscenes.py`, `validate_nuscenes.py` — nuScenes [1] training/validation, reporting AP/mATE/mASE/mAOE/NDS; `train_nuscenes.py --center` / `validate_nuscenes.py --center` already select the CenterPoint path.
- Trained checkpoint `checkpoint_nuscenes_center.pth` (CenterPoint head) for classes `Car, Pedestrian, Cyclist`. (`checkpoint_nuscenes.pth`, the anchor-head checkpoint, is superseded by this decision and is not part of the go-forward plan.)

Everything else in Fig. 2 of [6] — the tracking thread, the 2D detector, the fisheye/LiDAR fusion, the association logic, the shape reconstruction back-end, the asynchronous pipeline, and the joint optimizer — does not exist yet in this repository. The goal of this plan is to define, in order, what needs to be built to close that gap.

2 Gap Analysis

Table 1 maps every architectural component of DSP-SLAM++ to its status in `Code_tensor` today.

Paper component	**Paper ref.**	**Status**	**Notes**
3D object detection (PointPillars + CenterPoint head)	Sec. IV-A	**Present**	Present (`checkpoint_nuscenes_center.pth`), but (i) class set (`Car/Pedestrian/Cyclist`) does not match the target classes `car/bus/motorcycle/pedestrian`; (ii) per-class confidence thresholds $\tau_{c,3D}$ are not yet exposed per class in `center_postprocess.py`; (iii) `DetectionPipeline` (`pipeline.py`) currently instantiates the anchor-head model only and must be extended to load `CenterPointPillars` and call `center_postprocess`.
2D multi-class detection (YOLO11x-seg)	Sec. IV-A, VII-A	**Missing**	No 2D detector, no instance segmentation masks, no class labels for the image stream.

Paper component	Paper ref.	Status	Notes
Global greedy 2D–3D bipartite association	Sec. IV-A, Eqs. 1–3	**Missing**	Score matrix S, class-consistency gating, iterative arg-max assignment (Algorithm 1, lines 2–3) all absent.
Fisheye mask rectification	Sec. VI, VII-E	**Missing**	Contour-based rectification of predicted masks before projection scoring; absent.
Class-aware map association (radius r_c)	Sec. IV-B	**Missing**	Per-class matching radius from footprint diagonal, plus the second global re-association pass for loop closure/reappearance; absent.
Modular multi-decoder DeepSDF reconstruction	Sec. IV-C	**Missing**	No DeepSDF decoders, no per-class latent code optimization, no pose T_{co}/shape z_c refinement.
Asynchronous reconstruction back-end	Sec. V, Algorithm 2	**Missing**	Thread pool, task queue, result queue, bounding-box placeholder objects; absent.
Visual (fisheye) tracking front-end	Sec. III, VI	**Missing**	No ORB-SLAM3 [2] integration, no Kannala–Brandt [5] camera model, no keyframe/tracking thread.
RGB-L metric depth fusion	Sec. VI	**Missing**	LiDAR-to-feature depth association (feature-rectification strategy of [9]) absent; monocular scale ambiguity unresolved.
Joint map–object pose graph optimization	Sec. III	**Missing**	Camera poses, sparse map points, and object pose/shape jointly refined at each keyframe; absent.
Streaming dataset adapters (synced image+LiDAR+pose, calibration)	Sec. VII-A	**Partial**	`nuscenes_loader.py` exists but is a training-time loader (batched, shuffled); a time-synchronized, per-frame streaming loader with calibration/extrinsics for SLAM playback is absent, as are KITTI and custom fisheye-LiDAR adapters.
Evaluation harness (ATE, latency tables)	Sec. VII-B, D	**Missing**	No ATE trans/rot computation, no per-stage latency instrumentation (Detect./Object/Max Obj./KF BA latency).
Visualization (3D object map + trajectory)	Sec. VII-C, Fig. 7,9	**Partial**	`lidar_detection/visualization/visualizer.py` does BEV-only plots; 3D mesh map + trajectory overlay is absent.

Table 1: Component-by-component gap analysis against DSP-SLAM++.

3 Target Architecture

Figure 1 restates the DSP-SLAM++ thread architecture (Fig. 2 of [6]) and marks where the existing PointPillars pipeline plugs in versus what must be newly built.

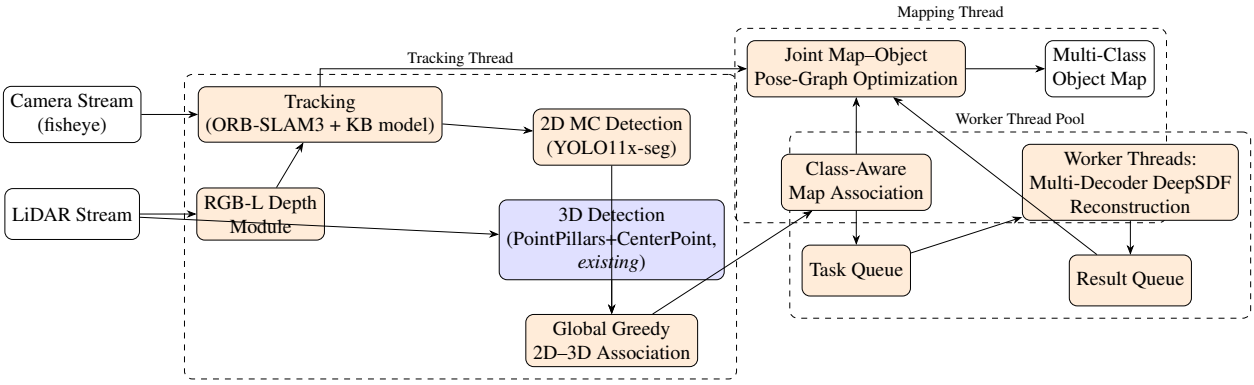


Figure 1: Target architecture. Blue = already implemented in `Code_tensor`. Orange = new work packages defined in Section 4.

4 Work Packages

Each work package (WP) lists its objective, the paper section it implements, its concrete deliverables, and which existing files it touches or depends on.

WP0 — Sensor, Calibration & Dataset Foundation

Objective: produce time-synchronized (image, LiDAR scan, pose-if-available) tuples with known intrinsics/extrinsics, for KITTI, nuScenes, and a custom fisheye-LiDAR rig.

- Streaming (non-shuffled, per-frame) adapters, separate from the existing training-time `nuscenes_loader.py`.
- Calibration containers: pinhole intrinsics for KITTI/nuScenes; Kannala–Brandt fisheye intrinsics [5] + camera-LiDAR extrinsics for the custom/CBNU-style rig.
- Ground-truth pose ingestion (GNSS/INS or KITTI/nuScenes poses) for later ATE evaluation.

Depends on: nothing (can start immediately). **Blocks:** WP2, WP3, WP9.

WP1 — 2D Multi-Class Detection & CenterPoint Wiring

Objective: produce D_{2D}: per-keyframe instance masks $M_{2D,j}$ with class labels $c_{2D,j}$ (Sec. IV-A); and make the existing 3D detector's CenterPoint path the one Algorithm 1 actually calls.

- Integrate a YOLO11x-seg [4] instance-segmentation model (or an equivalent pretrained segmenter) for the four target classes: car, bus, motorcycle, pedestrian.
- Class-specific confidence thresholds $\tau_{c,2D}$.
- Extend `DetectionPipeline` (`lidar_detection/pipeline.py`) with a `use_center` flag (mirroring `Trainer(cfg, use_center=True)`) so it instantiates `CenterPointPillars` and calls `center_postprocess` instead of the anchor path; load `checkpoint_nuscenes_center.pth` through this path.

- Expose per-class score thresholds $\tau_{c,3D}$ and per-class top-K in `center_postprocess.py` (currently a single global `score_threshold`/`top_k`).
- Extend/retrain the CenterPoint head's class set from `Car/Pedestrian/Cyclist` to `car/bus/motorcycle/pedestrian` (`lidar_detection/config.py` class list, `center_head.py` per-task heatmap channels, `center_loss.py`; retrain via `train_nuscenes.py --center`).

Depends on: WP0. **Blocks:** WP4.

WP2 — Fisheye Visual Tracking Front-End

Objective: robust camera pose estimation and keyframe selection under wide-angle distortion (Sec. III, VI).

- Integrate ORB-SLAM3 [2] (its native fisheye/unified camera model support) as the tracking backbone, via its existing C++ core with a Python-callable interface (pybind11), mirroring the approach used by the original DSP-SLAM [10].
- Keyframe decision logic feeding the object-detection stage.
- Sparse map-point management shared with the joint optimizer (WP8).

Depends on: WP0. **Blocks:** WP3, WP4, WP8.

WP3 — RGB-L Metric Depth Fusion

Objective: resolve monocular scale ambiguity by fusing LiDAR depth into the fisheye tracking pipeline (Sec. VI).

- Implement the *feature-rectification* strategy: undistort each extracted 2D feature via the Kannala–Brandt model into a virtual pinhole space, then project the LiDAR point cloud into that space to associate metric depth (cheaper than distorting the dense point cloud, per Fig. 5/Sec. VI of [6]).
- Adapt the lightweight RGB-L method [9] (originally pinhole-only) to consume rectified fisheye features.
- Feed metric depth into ORB-SLAM3's tracking to eliminate scale drift (reproduce the qualitative comparison in Fig. 6 of the paper).

Depends on: WP2. **Blocks:** WP9 (tracking-accuracy evaluation).

WP4 — Global Greedy 2D–3D Association

Objective: implement Algorithm 1 Stage 1 exactly (Sec. IV-A, Eqs. 1–3).

- Class-consistent score matrix $S \in \mathbb{Z}^{n_{3D} \times n_{2D}}$ using projected-point-in-mask ratios; -1 for cross-class pairs.
- Fisheye mask rectification (contour-extraction + KB rectification + polygon refill, Sec. VII-E) so masks and LiDAR projections live in a consistent space — this must run *before* scoring for fisheye datasets.
- Iterative arg-max assignment with row/column invalidation until all valid scores are exhausted (Eqs. 2–3).
- Wraps `lidar_detection.pipeline.DetectionPipeline.run()`, instantiated with `use_center=True` and `checkpoint_nuscenes_center.pth` (WP1), as the Det3D call in Algorithm 1 line 2, paired with the WP1 2D detector as Det2D.

Depends on: WP1, WP2 (for rectified projection geometry), WP3 (metric LiDAR-to-image association). **Blocks:** WP5.

WP5 — Class-Aware Map Association

Objective: implement Algorithm 1 Stage 2 (Sec. IV-B).

- Per-class matching radius $r_c = \sqrt{l_c^2 + w_c^2}$ from average class footprint.
- Ground-plane nearest-match association of validated detections against tracked map objects.
- Second global re-association pass against *all* map objects (not just currently tracked ones) to catch reappearance after tracking loss / loop closure, preventing duplicate instances.
- New-object instantiation for remaining unassociated candidates (Algorithm 1 lines 12–16), which triggers WP6/WP7.

Depends on: WP4. **Blocks:** WP6, WP7.

WP6 — Class-Specific Shape Reconstruction (Multi-Decoder DeepSDF)

Objective: implement Sec. IV-C: per-class DeepSDF [8] decoders $\{G_c\}_{c \in \mathcal{C}}$, optimized jointly for object pose T_{co} and shape code z_c.

- Train/obtain one DeepSDF decoder per class (car, bus, motorcycle, pedestrian) on ShapeNet [3] assets (plus any additional bus/motorcycle meshes), with consistent canonical orientation preprocessing.
- Runtime decoder dispatch keyed on the object's associated class label.
- Pose/shape optimization loop consuming the object's accumulated point cloud + 2D mask (same objective as the DSP-SLAM baseline [10]), producing an optimized latent code, pose, and extracted mesh.
- Document VRAM budget per decoder ($\sim$7 MB reported in the paper) to keep the multi-class design scalable.

Depends on: WP5 (needs class-labeled associated observations). **Blocks:** WP7.

WP7 — Asynchronous Reconstruction Back-End

Objective: implement Algorithm 2 (Sec. V) so reconstruction never blocks tracking/mapping.

- Two-worker thread pool (or process pool, see Section 6), thread-safe `TaskQueue` / `ResultQueue`.
- Mapping thread as *producer*: on new/updated objects, push a reconstruction task (point cloud, mask, pose) and immediately insert a bounding-box *placeholder* object into the map (instantaneous insertion).
- Worker(s) as *consumer*: run WP6's DeepSDF optimization + mesh extraction, push results.
- Mapping thread drains `ResultQueue` non-blockingly each cycle and swaps placeholders for optimized meshes.
- Priority rule: detection/association work pre-empts queued reconstruction tasks.

Depends on: WP5, WP6. **Blocks:** WP9 (latency benchmarking).

WP8 — Joint Map–Object Pose-Graph Optimization

Objective: jointly refine camera trajectory, sparse map points, and object poses/shape codes at each keyframe (Sec. III), matching ORB-SLAM3's local bundle adjustment plus the DSP-SLAM-style object terms.

- Extend ORB-SLAM3's local BA graph (g2o) with object pose/shape residuals.
- Ensure the async back-end (WP7) does not stall local BA – this is the metric evaluated as "KF BA Latency" in Table II of the paper.

Depends on: WP2, WP7. **Blocks:** WP9.

WP9 — Evaluation Harness

Objective: reproduce the paper's evaluation methodology (Sec. VII-B, D) on our own data.

- ATE (translation/rotation) computation against GT trajectories, mean/std over multiple runs (Table I style).
- Per-stage latency instrumentation: Detection latency, Object latency, Max Object latency, Keyframe BA latency, System FPS, Map object count (Table II style), comparing synchronous vs. asynchronous (WP7) modes.
- Extend `lidar_detection/evaluation/` with SLAM-level (not just per-frame detection) metrics.

Depends on: WP3, WP7, WP8. **Blocks:** nothing (terminal validation WP).

WP10 — Visualization

Objective: reproduce Fig. 7/9-style outputs: 3D multi-class object map with mesh geometry and trajectory overlay, plus mask-rectification diagnostic views (Fig. 8 style).

- Extend `lidar_detection/visualization/visualizer.py` (currently BEV-only) with a 3D mesh + trajectory renderer.

Depends on: WP6, WP8. **Blocks:** nothing.

5 Sequencing and Dependency Graph

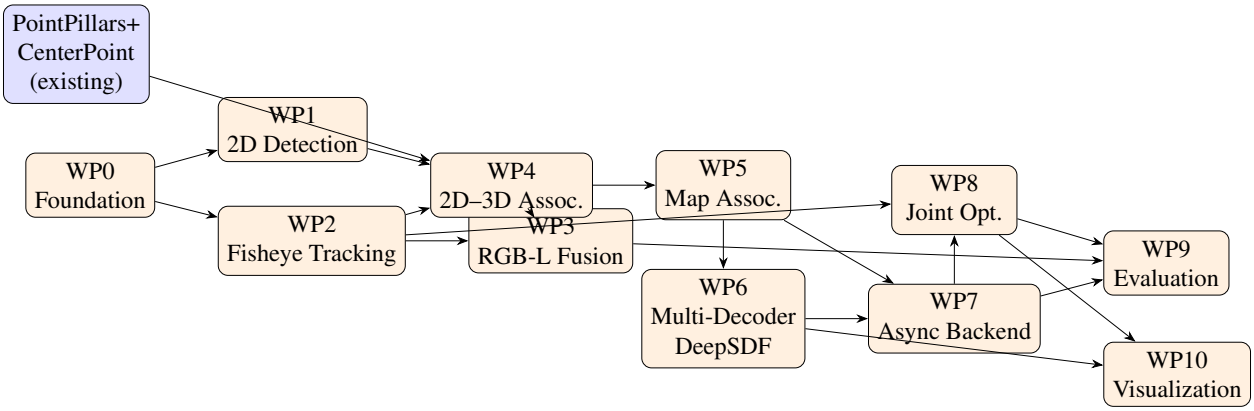


Figure 2: Work-package dependency graph. WP0/WP1/WP2 can start in parallel; WP6 (DeepSDF decoder training) can also be trained offline in parallel with WP2–WP4 since it only needs class labels and ShapeNet assets, not a working SLAM front-end.

Suggested milestones:

1. **M1 — Perception ready:** WP0, WP1, WP4 complete; class-labeled, associated 2D–3D detections available offline on recorded sequences (no tracking yet).
2. **M2 — Metric visual tracking ready:** WP2, WP3 complete; fisheye camera produces metric-scale trajectories comparable to Table I.
3. **M3 — Static object mapping:** WP5, WP6 complete; single-threaded (synchronous) reconstruction produces class-correct meshes anchored in the map.
4. **M4 — Real-time system:** WP7, WP8 complete; asynchronous pipeline meets target FPS with bounded KF BA latency.

5. **M5 — Validated system:** WP9, WP10 complete; ATE and latency tables reproduced on KITTI/nuScenes/custom data.

6 Architectural Decisions and Risks

- **Language boundary (C++ vs. Python).** ORB-SLAM3 is a C++ library; the current repository is pure Python/PyTorch. The original DSP-SLAM bridges this with pybind11 (C++ SLAM core, Python shape-optimization callbacks). Recommend following the same pattern rather than re-implementing a SLAM front-end in Python, to avoid re-deriving mature, well-tested tracking/BA code. This should be decided *before* WP2 starts, since it determines the entire process/threading model.

- **True parallelism for the async back-end (WP7).** Algorithm 2 assumes worker threads run genuinely in parallel with the mapping thread. If the DeepSDF optimization runs in Python, CPython's GIL will serialize pure-Python work; heavy tensor ops release the GIL during PyTorch's C++/CUDA execution, which likely suffices, but this should be measured early (a small prototype benchmark) rather than assumed. A process pool (via `multiprocessing`) is the fallback if thread-based concurrency proves insufficient.

- **Class taxonomy mismatch.** `checkpoint_nuscenes_center.pth` (the CenterPoint head selected for this plan) is trained for `Car/Pedestrian/Cyclist`; the target taxonomy is `car/bus/motorcycle/pedestrian`. Retraining (or fine-tuning) the CenterPoint head, and sourcing/curating bus and motorcycle instances in the training data, is on the critical path for WP1 and should be scheduled early given training time.

- **CenterPoint-only inference path.** Choosing the CenterPoint head as the sole go-forward detector means the anchor head (`PointPillars`, `postprocess.py`, `anchor.py`, `nms.py`) becomes dead code for this project; it can stay in the repository for reference/comparison but should not be a dependency of any new WP. `DetectionPipeline` additionally needs the `use_center` inference wiring called out in WP1 before it can load `checkpoint_nuscenes_center.pth` at all.

- **DeepSDF training data for non-standard classes.** ShapeNet has strong car coverage but weaker/no dedicated bus or motorcycle categories at the same fidelity; asset curation and canonical-orientation preprocessing for these classes is a real risk to reconstruction quality and should be scoped during WP6 planning, not discovered mid-implementation.

- **Licensing.** ORB-SLAM3 is GPLv3-licensed; any distribution of the integrated system inherits that obligation. Worth flagging now if the project has different distribution plans.

- **Sensor rig availability.** WP3 (fisheye + LiDAR metric fusion) needs real or simulated fisheye-LiDAR data with known extrinsics; if no such rig/dataset is available in-house yet, nuScenes (pinhole) can validate WP4–WP9 first, deferring true fisheye validation to whatever dataset is used to reproduce Table I's fisheye rows (CBNU-style or custom).

- **Evaluation dataset gaps.** Table I/II reproduction requires GT trajectories (GNSS/INS or dataset-provided poses); confirm which target datasets actually have usable ground truth before committing to WP9's exact metrics.

7 Proposed Repository Additions

To keep the existing, working `lidar_detection` package untouched as the Det3D component, new work is proposed as sibling top-level packages:

```
Code_tensor/
|-- lidar_detection/      # existing, unchanged (Det3D)
|-- detection2d/          # WP1: YOLO11x-seg wrapper, class thresholds
|-- slam_frontend/        # WP2/3: ORB-SLAM3 bridge, RGB-L fusion, KB rectification
|-- association/          # WP4/5: 2D-3D matching, class-aware map association
|-- reconstruction/       # WP6: multi-decoder DeepSDF, pose/shape optimization
|   |-- decoders/         #   per-class pretrained DeepSDF weights
|-- async_backend/        # WP7: task/result queues, worker pool, placeholders
|-- evaluation_slam/      # WP9: ATE + latency benchmarking (SLAM-level;
|                         #   existing lidar_detection/evaluation is per-frame only)
|-- datasets/             # WP0: streaming KITTI/nuScenes/custom adapters + calibration
```

This keeps the pretrained detector reusable as-is (imported by `association/` as the Det3D call), while isolating the large amount of new, cross-cutting SLAM infrastructure.

8 Summary

The repository today implements roughly one of the thirteen components listed in Table 1 (3D detection, standardized on the CenterPoint head and `checkpoint_nuscenes_center.pth`), plus a partial second (streaming dataset support). The remaining work — a fisheye-capable visual tracking front-end, LiDAR metric-depth fusion, multi-class 2D detection, the class-consistent global association algorithm, per-class DeepSDF reconstruction, the asynchronous back-end, and joint pose-graph optimization — is substantial and mostly independent of the existing PointPillars/CenterPoint code, which plugs in cleanly at a single, well-defined point (Algorithm 1, line 2) once the small `DetectionPipeline` wiring change in WP1 is made. The work packages above are ordered so that perception (WP0/1/4), tracking (WP2/3), and offline shape-model training (WP6) can proceed in parallel, converging on the asynchronous, jointly-optimized system (WP7/8) that the evaluation harness (WP9) then validates against the paper's own metrics.

References

[1] Holger Caesar, Varun Bankiti, Alex H. Lang, Sourabh Vora, Venice Erin Liong, Qiang Xu, Anush Krishnan, Yu Pan, Giancarlo Baldan, and Oscar Beijbom. nuScenes: A multimodal dataset for autonomous driving. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 11621–11631, 2020.

[2] Carlos Campos, Richard Elvira, Juan J. G. Rodríguez, José M. M. Montiel, and Juan D. Tardós. ORB-SLAM3: An accurate open-source library for visual, visual-inertial, and multi-map SLAM. *IEEE Transactions on Robotics*, 37(6):1874–1890, 2021.

[3] Angel X. Chang, Thomas Funkhouser, Leonidas Guibas, Pat Hanrahan, Qixing Huang, Zimo Li, Silvio Savarese, Manolis Savva, Shuran Song, Hao Su, Jianxiong Xiao, Li Yi, and Fisher Yu. ShapeNet: An information-rich 3d model repository. Technical Report arXiv:1512.03012 [cs.GR], Stanford University — Princeton University — Toyota Technological Institute at Chicago, 2015.

[4] Glenn Jocher and Jing Qiu. Ultralytics YOLO11. https://github.com/ultralytics/ultralytics, 2024.

[5] Juho Kannala and Sami S. Brandt. A generic camera model and calibration method for conventional, wide-angle, and fish-eye lenses. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 28(8):1335–1340, 2006.

[6] Ahmad Kourani, Ghina Daoud, Daniel Asmar, and Imad Elhajj. DSP-SLAM++: A unified framework for multi-class, high-fidelity object SLAM in the wild. *arXiv preprint arXiv:2606.25953*, 2026.

[7] Alex H. Lang, Sourabh Vora, Holger Caesar, Lubing Zhou, Jiong Yang, and Oscar Beijbom. PointPillars: Fast encoders for object detection from point clouds. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 12697–12705, 2019.

[8] Jeong Joon Park, Peter Florence, Julian Straub, Richard Newcombe, and Steven Lovegrove. DeepSDF: Learning continuous signed distance functions for shape representation. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 165–174, 2019.

[9] Fabian Sauerbeck, Benedikt Obermeier, Martin Rudolph, and Johannes Betz. RGB-L: Enhancing indirect visual SLAM using LiDAR-based dense depth maps, 2022.

[10] Jingwen Wang, Martin Rünz, and Lourdes Agapito. DSP-SLAM: Object oriented SLAM with deep shape priors. In *2021 International Conference on 3D Vision (3DV)*, pages 1362–1371. IEEE, 2021.

[11] Tianwei Yin, Xingyi Zhou, and Philipp Krähenbühl. Center-based 3d object detection and tracking. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 11784–11793, 2021.